

コンテナの基本

目次

第1章 コンテナの基本	3
イメージとコンテナ	4
ポートを公開して接続する	5
イメージを共有して動かす	6
索引	7

コンテナの基本

この章で学ぶこと

- ・イメージとコンテナの違いを説明する
- ・ホストとコンテナのポートの対応を読む
- ・イメージを共有して別の場所で起動する流れを追う

1.1 イメージとコンテナ

実行のもとと、実行する環境を区別する。

アプリを動かすには、プログラムだけでなく、実行環境や必要なファイルも揃える必要があります。これらをまとめたものが、**イメージ (Image)** です。

イメージをもとに作られる実行環境を、**コンテナ (Container)** と呼びます。同じイメージから、複数のコンテナを起動できます。

用語の補足

イメージは実行のもとになるもの。コンテナは、そのイメージから作る個々の実行環境です。

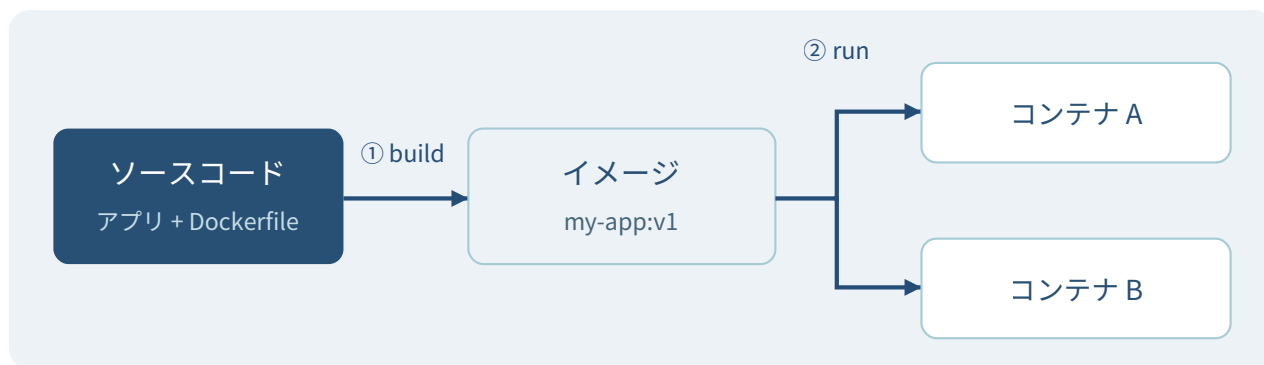


図 1.1 1つのイメージから、複数のコンテナを起動できる

1.1.1 操作を、対象と結び付ける

コマンドは「何をもとに、何をするか」で整理できます。イメージを作る操作と、コンテナを起動する操作は別です。

表 1.1 コマンドと操作する対象

コマンド	対象	結果
<code>docker build</code>	ソースと Dockerfile	イメージを作る
<code>docker run</code>	イメージ	コンテナを作って起動
<code>docker push</code>	イメージ	レジストリへ送る

POINT 送るのは、イメージ

別の場所でアプリを動かすときは、イメージを共有し、その場所で新しいコンテナを起動します。動いているコンテナそのものを送るわけではありません。

1.2 ポートを公開して接続する

「動いている」と「接続できる」は、別の確認。

コンテナ内でアプリが動いていても、ホストから接続できるとは限りません。ホスト側のポートとコンテナ側のポートを対応付けて、接続先を用意します。

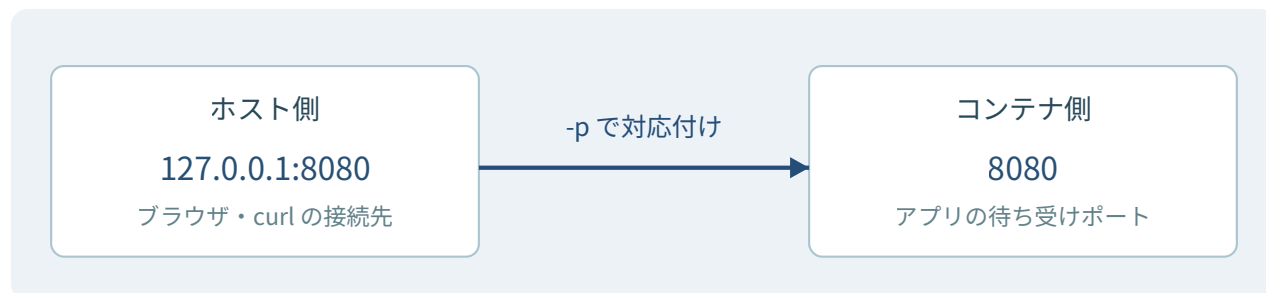


図 1.2 ホストのローカルアドレスから、コンテナ内のアプリへ接続する

1.2.1 起動コマンドを読む

ローカル接続用のポートを公開して起動

```
docker run -d --name demo \
  -p 127.0.0.1:8080:8080 my-app:v1
```

`my-app:v1` は使用するイメージ、`demo` はコンテナの名前です。`-p` の指定は、ホスト側のアドレス、ホスト側のポート、コンテナ側のポートの順に並びます。

表 1.2 ポートに関する設定の違い

設定	役割
<code>EXPOSE</code>	アプリが使用するポートの情報を示す
<code>docker run -p</code>	ホスト側とコンテナ側のポートを対応付けて公開する

POINT 情報と公開設定を区別する

Dockerfile に `EXPOSE` を記述しただけでは、ホスト側へのポート公開は行われません。実際の公開には `-p` などの指定が必要です。

前提：イメージは作成済み。アプリはコンテナ内の `0.0.0.0:8080` で待ち受けます。掲載例は実行未検証です。

1.3 イメージを共有して動かす

同じアプリを、別の場所で新しく起動する。

ローカルで作ったアプリを別の場所で動かすときは、イメージをレジストリへ送ります。配備先では、そのイメージを取得し、新しいコンテナを起動します。

この流れを「作成」「保存・共有」「起動」に分けると、どの対象がどこへ渡るのかを整理できます。

レジストリとは

イメージを保管して共有する場所です。Google Cloud では Artifact Registry などを利用します。

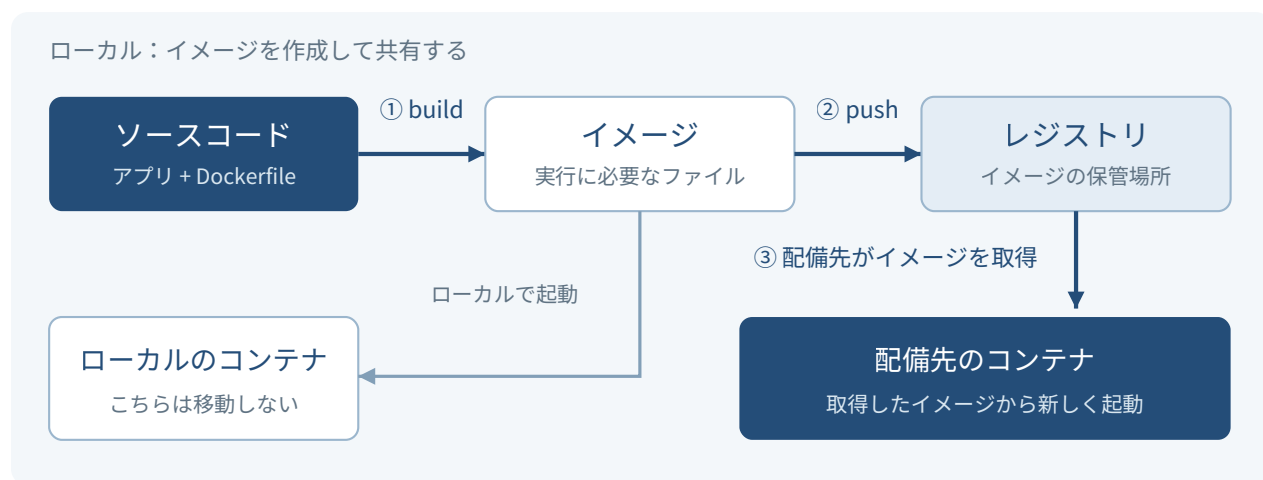


図 1.3 イメージの流れと、ローカル・配備先の実行環境

1.3.1 実行環境は、それぞれ独立する

同じイメージを使っている、コンテナは別々の実行環境です。一方を停止したという理由だけで、もう一方も停止するわけではありません。

イメージが同じでも、実行中の状態は別々



図 1.4 同じイメージから作ったコンテナでも、実行状態はそれぞれ別

POINT 共有するものと、別々に動くもの

共有するのはイメージです。コンテナは各環境で作って動かします。基本の関係は（→ p.4）を参照してください。

索引

E

EXPOSE 5

あ

イメージ 4

か

コンテナ 4